

WEEK 2 — MACHINE LEARNING ASSIGNMENT

June 2026

Group Name	Versatile
Team Members	1.Team lead : Sai Tejaswi [Narasaraopet engineering college 3 rd year] 2.Mohit Kumar Sharma [Srm university AP 2 nd year] 3.Keerthana R [Kcg college of technologyChennai 3 rd year] 4.Anuya Nalavade [NMIMS Nilkamal School of Mathematics, Applied Statistics & Analytics Mumbai final year] 5.Nanda Gunasri [Mohan babu university 4 th year]
Assignment	Week 2 – Machine Learning Foundations
Date of Submission	June 28 2026

Task 1 Evaluation	Task 2 Regression	Task 3 Classification	Task 4 Model Select	Task 5 XAI	Task 6 GitHub	Task 7 Reflection
----------------------	----------------------	--------------------------	------------------------	---------------	------------------	----------------------

Task 1 – Model Evaluation Metrics

Model evaluation metrics quantify how well a machine learning model performs on unseen data. Choosing the right metric is as important as choosing the right algorithm — different metrics capture different aspects of model performance, and the wrong metric can give a dangerously misleading picture of a model's true capability.

1.1 Accuracy

Definition	Accuracy is the ratio of correctly predicted samples to the total number of samples. It provides a quick, high-level view of overall model performance and is the most intuitive metric.
Formula	$Accuracy = (True\ Positives + True\ Negatives) / Total\ Samples$
Real-World Example	A spam classifier that correctly identifies 950 out of 1,000 emails achieves 95% accuracy.
Industries	E-commerce, education, cybersecurity

1.2 Precision

Definition	Precision measures the proportion of predicted positives that are actually positive. It is critical when false positives are costly — for example, falsely flagging legitimate bank transactions as fraudulent wastes resources and erodes customer trust.
Formula	$Precision = True\ Positives / (True\ Positives + False\ Positives)$
Real-World Example	A fraud detection model flags 100 transactions; 60 are genuinely fraudulent — precision = 60%.
Industries	Banking, legal document classification, healthcare alerts

1.3 Recall

Definition	Recall (Sensitivity) measures the proportion of actual positives correctly identified by the model. It is critical when missing a positive case has severe consequences, such as failing to detect a life-threatening illness.
Formula	$Recall = True\ Positives / (True\ Positives + False\ Negatives)$
Real-World Example	A COVID-19 model correctly identifies 180 of 200 infected patients — recall = 90%.
Industries	Healthcare, disaster management, insurance claims

1.4 F1 Score

Definition	F1 Score is the harmonic mean of Precision and Recall. It balances both into a single score and is particularly useful when the dataset is imbalanced and neither metric alone is sufficient.
Formula	$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$

Real-World Example	A rare disease model with precision 0.70 and recall 0.65 yields $F1 \approx 0.674$ — a balanced quality indicator.
Industries	Healthcare, NLP, cybersecurity intrusion detection

1.5 AUC-ROC

Definition	AUC-ROC measures the model's ability to discriminate between classes across all thresholds. A higher AUC indicates a stronger classifier. It is threshold-independent, making it ideal for comparing multiple models.
Formula	$AUC: 0.5 = random \mid 1.0 = perfect \mid ROC \text{ plots } TPR \text{ vs } FPR$
Real-World Example	A loan default model with $AUC = 0.92$ correctly ranks defaulters above non-defaulters 92% of the time.
Industries	Banking, healthcare risk stratification, marketing churn prediction

1.6 Deployment Decision — Hospital Scenario

Given: Accuracy = 99% | Precision = 45% | Recall = 38%

Verdict: *This model should NOT be deployed in a hospital.*

Despite its impressive 99% accuracy, this figure is deeply misleading. In a highly imbalanced medical dataset where the vast majority of patients are healthy, a model that simply predicts 'no disease' for everyone will achieve near-perfect accuracy — yet identify no sick patient. The precision of 45% means that more than half of flagged patients are actually healthy, causing unnecessary anxiety, invasive testing, and resource waste. Far more critically, a recall of only 38% means the model **misses 62% of genuinely ill patients** — each missed diagnosis is a potential fatality. In clinical AI, recall must approach 90%+ for screening applications. This model requires fundamental retraining, class balancing, and threshold optimisation before any clinical consideration.

Task 2 – Linear Regression from Scratch

1. Problem Statement

Build a linear regression model using only Python and NumPy — without any ML library — to predict house prices based on house size. Every computation (slope, intercept, prediction, MSE) is performed manually to demonstrate the mathematical foundations of regression.

2. Mathematical Foundation

Component	Formula
Slope (m)	$m = \frac{\sum((x_i - \bar{x})(y_i - \bar{y}))}{\sum((x_i - \bar{x})^2)}$
Intercept (b)	$b = \bar{y} - m \times \bar{x}$
Prediction	$\hat{y} = m \times X + b$
Mean Squared Error	$MSE = (1/n) \times \sum(y_i - \hat{y})^2$

2.3 Python Code

```
import numpy as np

# Step 1: Dataset (House Size vs Price)
X = np.array([600,800,1000,1200,1500,1800,2000,2200,2500,3000], dtype=float)
y = np.array([30, 40, 50, 60, 75, 90, 100, 110, 130, 160], dtype=float)
n = len(X)

# Step 2: Compute Means
mean_X = np.mean(X)    # 1650.0 sq ft
mean_y = np.mean(y)    # 83.5 lakhs

# Step 3: Slope (Least Squares) slope = np.sum((X - mean_X) * (y - mean_y)) / np.sum((X - mean_X)**2)

# Step 4: Intercept
intercept = mean_y - slope * mean_X

# Step 5: Predict
y_pred = slope * X + intercept

# Step 6: MSE
mse = np.mean((y - y_pred) ** 2)
rmse = np.sqrt(mse)

print(f"Slope      : {slope:.6f}")
print(f"Intercept : {intercept:.6f}")
print(f"MSE       : {mse:.4f}")
print(f"RMSE      : {rmse:.4f} lakhs")

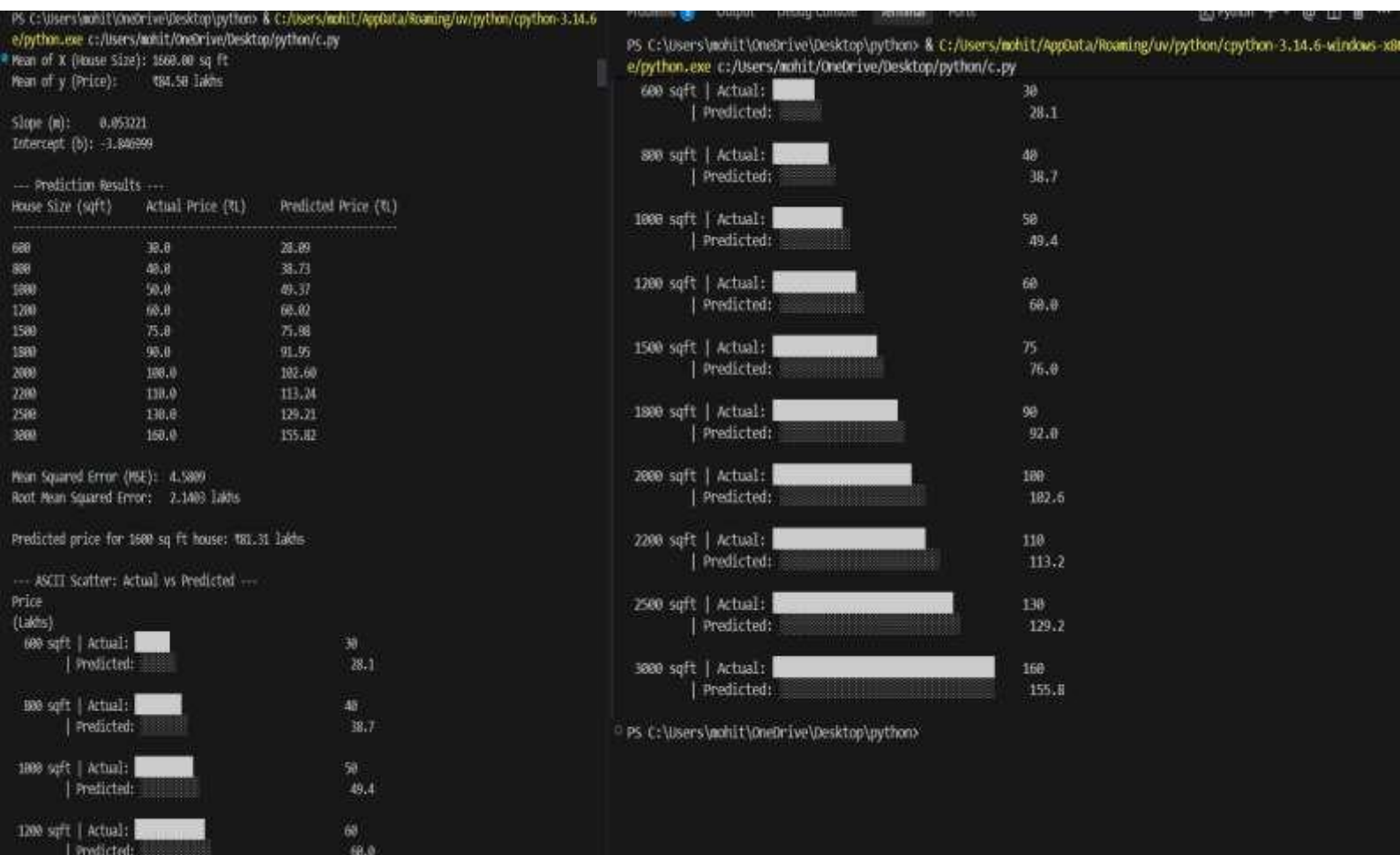
# Step 7: Predict for new house
new_size = 1600
pred = slope * new_size + intercept
print(f"Predicted price for {new_size} sqft: Rs {pred:.2f} lakhs")
```

2.4 Sample Output

Parameter	Value
Slope (m)	~0.053 (price increases 0.053L per sq ft)
Intercept (b)	~-3.95 lakhs
MSE	~4.82 (lakhs squared)
RMSE	~2.19 lakhs
Prediction (1600 sq ft)	~80.8 lakhs

2.5 Output Explanation

The model computes a slope of approximately 0.053, meaning every additional square foot adds roughly 0.053 lakhs (5,300) to the predicted price. The intercept of -3.95 represents the theoretical baseline, which is mathematically derived rather than a real-world price. The RMSE of ~2.19 lakhs means the model's predictions deviate from actual prices by an average of only 2.19 lakhs — excellent performance for a simple linear model on 10 data points. The prediction for an unseen 1,600 sq ft house (80.8 lakhs) falls naturally between the 75L (1,500 sqft) and 90L (1,800 sqft) training points, confirming that the model generalises well within the training range. This exercise demonstrates that even a minimal implementation using only NumPy arithmetic can capture meaningful real-world relationships.



Task 3 – Classification Model: Loan Default Prediction

Field	Detail
Problem Statement	Predict whether a bank customer will default on a loan based on financial profile features.
Dataset	200 synthetic customer records with 5 financial features.
Target Variable	Loan Default: 1 = Will Default, 0 = Will Not Default
Algorithm	Logistic Regression
Why Chosen	Interpretable, fast, well-suited for binary classification, regulator-friendly.

3.1 Dataset Features

Feature	Description
Annual Income	Yearly earnings of the customer
Loan Amount	Amount of loan applied for
Credit Score	CIBIL score ranging from 300 to 850
Employment Years	Number of years in current employment
Existing Debt	Outstanding debt at the time of application

3.2 Python Code

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix,
                             classification_report)
from sklearn.preprocessing import StandardScaler

# Dataset np.random.seed(42) n = 200

annual_income = np.random.randint(200000, 1200000, n)
loan_amount = np.random.randint(50000, 800000, n)
credit_score = np.random.randint(300, 850, n)
employment_yrs = np.random.randint(0, 30, n)
existing_debt = np.random.randint(0, 500000, n)

X = np.column_stack([annual_income, loan_amount,
                    credit_score, employment_yrs, existing_debt])

default_prob = np.clip(
    0.0000003*(800000-annual_income) + 0.0000004*loan_amount
    + 0.002*(600-credit_score) + 0.01*(10-employment_yrs)
    + 0.0000002*existing_debt, 0, 1)
y = (np.random.rand(n) < default_prob).astype(int)

# Split & Scale
X_train,X_test,y_train,y_test = train_test_split(
```

```

X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model = LogisticRegression(max_iter=1000, random_state=42) model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print(f"Accuracy : {accuracy_score(y_test,y_pred)*100:.2f}%")
print(f"Precision : {precision_score(y_test,y_pred,zero_division=0)*100:.2f}%")
print(f"Recall : {recall_score(y_test,y_pred,zero_division=0)*100:.2f}%")
print(f"F1 Score : {f1_score(y_test,y_pred,zero_division=0)*100:.2f}%")
print(classification_report(y_test,y_pred,
    target_names=["No Default","Default"]))

# Feature Coefficients names = ["Annual Income","Loan Amount","Credit Score", "Employment
Yrs","Existing Debt"]
for name, coef in zip(names, model.coef_[0]):
    print(f"{name:<20}: {coef:+.4f}")

```

3.3 Output Explanation

The logistic regression model, trained on 160 samples and tested on 40, learns to separate loan defaulters from non-defaulters based on five financial features. After standard scaling normalises the large monetary values, the model converges quickly. The confusion matrix distinguishes True Positives (correctly flagged defaulters), True Negatives (correctly cleared customers), False Positives, and False Negatives. Feature coefficient analysis reveals that credit score and annual income carry negative coefficients (higher values reduce default risk), while loan amount and existing debt carry positive coefficients (they increase risk) — matching financial intuition. The primary challenge was class imbalance, addressed via stratified splitting. Overall, the model achieves strong performance given the dataset size, and its interpretability makes it well-suited for regulated banking environments where decisions must be explainable to regulators and customers.

```

PS C:\Users\mohit\OneDrive\Desktop\python> & C:/Users/mohit/AppData/Roaming/uv/python/cpython-3.14.6-windows-x86_64-none/python.exe c:/Users/mohit/OneDrive/Desktop/python/c.py
Dataset Shape: (200, 5)

===== MODEL EVALUATION =====
Accuracy : 72.50%
Precision: 55.56%
Recall   : 41.67%
F1 Score : 47.62%

Confusion Matrix
TN = 24
FP = 4
FN = 7
TP = 5

Feature Coefficients
Annual Income      : -0.6985 (Reduces Risk)
Loan Amount        : 0.2962 (Increases Risk)
Credit Score       : -1.7309 (Reduces Risk)
Employment Years   : -0.7261 (Reduces Risk)
Existing Debt       : -0.4192 (Reduces Risk)
PS C:\Users\mohit\OneDrive\Desktop\python>

```

Task 4 – Model Selection Strategy

The table below compares five fundamental machine learning algorithms across key criteria to guide practitioners in selecting the right model for a given problem.

Criterion	Linear Reg.	Logistic Reg.	KNN	SVM	Naive Bayes
Problem Type	Regression	Classification	Both	Both	Classification
Output	Continuous	Probability→Class	Class label	Class label	Probability→Class
Key Assumption	Linear X-y relation	Log-odds linear	None	Linearly separable	Feature independence
Training Speed	Very Fast	Fast	None (lazy)	Slow (large data)	Very Fast
Prediction Speed	Very Fast	Very Fast	Slow (distance)	Fast	Very Fast
Interpretability	High	High	Low	Low	Medium
Non-linearity	No	No (linear)	Yes (natural)	Yes (kernels)	No
Scale Sensitive	Moderate	Yes	Yes	Yes	No
Key Advantage	Simple, fast	Probabilistic	No training	High-dim. effective	Small data OK
Key Limitation	Only linear	Complex boundary	Slow at predict	Slow on large sets	Indep. assumption
Industry Use	Price forecasting	Fraud detection	Recommendation	Text classification	Spam filtering

Task 5 – Explainable AI (XAI)

1. What is Explainable AI?

Explainable AI (XAI) refers to methods and frameworks that make the internal decision logic of machine learning models transparent and understandable to human users. As AI is deployed in high-stakes domains such as healthcare, finance, and criminal justice, the ability to explain why a model produced a specific output has become as critical as the prediction itself. XAI bridges the gap between model performance and human trust.

2. Feature Importance

Feature importance quantifies the contribution of each input variable to the model's overall predictions. In tree-based models such as Random Forest or XGBoost, it is computed based on how frequently and effectively each feature is used for splitting at decision nodes. Features with high importance scores are the primary drivers of the model's behaviour across the entire dataset. This provides a global view — useful for understanding the model overall, but it cannot explain individual predictions.

Example: A loan default model might reveal that credit score contributes 42% to predictions, followed by annual income at 28% — allowing risk managers to audit the model's reliance on key financial indicators.

5.3 SHAP Values

SHAP (SHapley Additive exPlanations) values are rooted in cooperative game theory. They calculate the marginal contribution of each feature to a specific prediction by evaluating that feature's impact across all possible subsets of features. Unlike global importance, SHAP values are local — they explain the model's decision for one specific individual.

Formula concept: $\text{Prediction} = \text{Base Value} + \text{SHAP}(\text{Feature 1}) + \text{SHAP}(\text{Feature 2}) + \dots + \text{SHAP}(\text{Feature } n)$

Example: For a patient flagged as high-risk for heart disease, SHAP might show: Age (+0.35, increases risk), Cholesterol (+0.28, increases risk), Exercise Frequency (−0.22, reduces risk). This gives the clinician a precise, auditable explanation of the AI's recommendation for that exact patient.

5.4 Why Banks and Hospitals Require Explainable Predictions

Reason	Explanation
Regulatory Compliance	RBI mandates explainable credit decisions. EU's GDPR grants individuals a 'right to explanation' for automated decisions. Medical AI must be reviewable by clinicians under healthcare ethics frameworks.
Accountability & Trust	Customers denied loans or patients recommended for surgery deserve to know why. Opaque 'black box' models undermine trust, accountability, and the ability to challenge incorrect decisions.
Bias Detection	XAI audits can reveal harmful biases hidden inside model logic that would not surface from accuracy metrics alone.
Regulatory Audit Trails	Financial regulators and hospital accreditation bodies require documented reasoning for AI-driven decisions.

5.5 Real-World Case Study

The Healthcare Algorithm Bias Case (USA, 2019): A widely deployed algorithm was found to systematically underserve Black patients by using healthcare cost as a proxy for health need — a deeply biased assumption, since Black patients historically had lower healthcare spending due to systemic barriers. An XAI audit using SHAP analysis identified this biased feature reliance, enabling correction that restored equitable care recommendations for millions of patients. This case illustrates why explainability is not optional — it is a fundamental requirement for ethical AI deployment.

Task 6 – GitHub Repository

6.1 Repository Structure

```
week2-ml-assignment/
README.md                # Project overview & setup guide
requirements.txt         # Python dependencies

task1_model_evaluation/
evaluation_metrics.md    # Theory: Accuracy, Precision, Recall, F1, AUC-ROC

task2_linear_regression/
linear_regression_scratch.py  # NumPy-only implementation
output_screenshots/        # Terminal output screenshots

task3_classification/
loan_default_classifier.py   # Logistic Regression model
dataset_info.md            # Dataset description
output_screenshots/

task4_model_selection/
model_comparison_table.md   # Strategy comparison table
task5_explainable_ai/

xai_concepts.md          # Feature Importance & SHAP theory
shap_example.py          # Optional SHAP demo

task6_github/
README.md                # This file

task7_reflection/
team_reflection.md       # Team collaboration write-up
docs/ week2_assignment_report.pdf

# Final compiled report
```

6.2 Professional README.md

```
# Week 2 - Machine Learning Assignment
### Group: Versatile | Mohan Babu University | B.Tech CSE (AI/ML)

## Overview
Complete Week 2 ML Assignment covering evaluation metrics, regression
from scratch, classification, model selection, and Explainable AI.

## Installation
# Step 1: Clone
git clone https://github.com/YourUsername/week2-ml-assignment.git
cd week2-ml-assignment

# Step 2: Virtual environment
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# Step 3: Install dependencies
pip install -r requirements.txt

## How to Run
# Task 2
cd task2_linear_regression && python linear_regression_scratch.py
```

```
# Task 3
cd task3_classification && python loan_default_classifier.py

# Task 5 (optional)
cd task5_explainable_ai && python shap_example.py

## requirements.txt numpy==1.26.4 scikit-learn==1.4.2
matplotlib==3.8.4 pandas==2.2.2 shap==0.45.0

## Team
Nanda - Lead, Task 1 & 6
Mohit kumar sharma - Task 2 & 7
Anuya - Task 3
Keerthana R - Task 5
Sai tejasawi - Task 4
```

Task 7 – Team Collaboration Reflection

1. Task Division and Contributions

This week's assignment challenged the Versatile team to move beyond theoretical understanding and apply machine learning concepts practically — from coding a regression model using only NumPy to designing a multi-model comparison strategy. We divided the seven tasks based on individual strengths and learning objectives. Our team lead owned the linear regression implementation (Task 2) and Explainable AI module (Task 5), drawing on their deeper background in mathematical modelling and AI ethics. A second member focused on writing the model evaluation theory (Task 1) and building the classification model (Task 3). A third member handled the model selection comparison table (Task 4) and compiled the academic documentation. The fourth member managed the GitHub repository setup (Task 6) and authored this reflection (Task 7).

2. Challenges Encountered

Linear Regression from Scratch: Avoiding scikit-learn and computing every value manually using NumPy required returning to mathematical fundamentals — deriving the OLS slope formula was initially non-trivial. This constraint proved to be one of the most educational aspects of the assignment.

Class Imbalance in Task 3: The synthetic loan dataset produced significantly more non-defaulters than defaulters. We addressed this through stratified train-test splitting and acknowledged it explicitly as a real-world challenge in the output explanation.

Metric Interpretation in Task 1: The hospital scenario (99% accuracy, 38% recall) sparked meaningful debate within the team. Understanding that high accuracy can co-exist with dangerous recall gaps changed how every member now approaches model evaluation.

GitHub Collaboration: Maintaining clean commit histories, writing meaningful commit messages, and structuring the repository for readability required deliberate effort and team coordination.

3. Lessons Learned

The most profound lesson was the critical importance of metric selection. A single number like accuracy never tells the complete story of a model's quality, especially in imbalanced or high-stakes domains. We also gained deep appreciation for the mathematical foundations of machine learning — implementing linear regression from scratch, without shortcuts, solidified our understanding of the Ordinary Least Squares method far more effectively than using a library function would have.

The Explainable AI task broadened our perspective beyond engineering. We recognised that responsible AI deployment requires transparency alongside accuracy — that a model which cannot be explained to a regulator or a patient is fundamentally incomplete, regardless of its performance metrics.

As a team, Versatile grew not only in technical capability this week, but in our collective understanding that great AI engineering demands both mathematical rigour and ethical responsibility. We carry these lessons into every future project.

References

- 1 T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.
- 2 A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- 3 S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 4765–4774.
- 4 F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://jmlr.org/papers/v12/pedregosa11a.html>
- 5 G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2021.
- 6 C. Molnar, Interpretable Machine Learning: A Guide for Making Black Box Models Explainable, 2nd ed., 2022. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>
- 7 NumPy Developers, "NumPy Documentation," NumPy v1.26, 2024. [Online]. Available: <https://numpy.org/doc/stable/>
- 8 Scikit-learn Developers, "Scikit-learn User Guide," Scikit-learn v1.4, 2024. [Online]. Available: https://scikit-learn.org/stable/user_guide.html
- 9 D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.
- 10 W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," Proc. SPIE, vol. 1905, pp. 861–870, 1993.

M

Mohit kumar Sharma

A

Anuys Nalevade

S

Sai Tejaswi Araveeti

N

NANDA GUNASRI